Analysis of Previous Studies

Study (1)

ReAct: Synergizing Reasoning and Acting in Language Models

This study, by Yao et al. (Princeton University and Google Research, ICLR 2023), starts from the problem that reasoning and acting in large language models have mainly been studied separately. Reasoning alone is not grounded in the external world, which leads to hallucination and error propagation.

The researchers propose ReAct, in which the model generates reasoning traces and actions in an interleaved manner. Reasoning helps the model create and update its action plans, while actions let it gather information from external sources such as a Wikipedia API.

Each task is solved as a sequence of thought–action–observation steps, using PaLM-540B prompted with only a few examples. The approach was tested on HotpotQA, FEVER, ALFWorld and WebShop, and fine-tuning on 3,000 examples let PaLM-8B outperform all PaLM-62B prompting methods.

ReAct outperformed imitation and reinforcement learning methods by an absolute success rate of 34% on ALFWorld and 10% on WebShop. Hallucination caused 0% of its failures, compared with 56% for chain-of-thought. However, the interleaved structure caused more reasoning errors, and complex tasks need more examples than the input length allows.

Study (2)

TinyAgent: Function Calling at the Edge

This study, by Erdogan et al. (UC Berkeley, EMNLP 2024), starts from the problem that large models are too big to run locally, so their inference runs in the cloud. This causes privacy issues, needs a stable connection and adds latency.

The researchers propose TinyAgent, a framework for training and deploying small language models that perform function calling on the device itself. As an application, they build a local Siri-like assistant for MacBook with 16 functions, such as email, calendar and notes.

They generated 80K training examples with GPT-4-Turbo and fine-tuned TinyLlama-1.1B and Wizard-2-7B with LoRA. They also added Tool RAG, which selects only the tools a query needs, and 4-bit quantization to make the models smaller and faster.

Fine-tuning raised the 1.1B model's success rate from 12.71% to 78.89%. The final models reached 80.06% (1.1B) and 84.95% (7B), above GPT-4-Turbo (79.08%), and the 1.1B model ran locally at only 0.68 GB. However, the authors note that the risk of hallucination remains, and that synthetic data may carry cultural biases.

Study (3)

FinStat2SQL: A Text2SQL Pipeline for Financial Statement Analysis

This study, by Nguyen et al. (National Economics University, Vietnam, INLG 2025), starts from the problem that Text2SQL remains difficult for complex financial queries. Vietnam's local accounting standard (VAS) differs from IFRS, which adds more complexity to querying financial data.

The researchers propose FinStat2SQL, a multi-agent pipeline that lets users query financial statements in natural language. It combines large and small language models for entity extraction, SQL generation and self-correction.

They built a database of 200 Vietnamese companies, generated 12,400 synthetic SQL training samples with large models, and fine-tuned Qwen 2.5 Coder models (1.5B, 3B and 7B) with LoRA. The evaluation used about 300 real financial questions, and the researchers checked the overlap between the synthetic and evaluation data.

The fine-tuned 7B model reached 61.33% accuracy with under 4 seconds of latency, outperforming GPT-4o mini (59.14%) but not Llama 3.3 70B (65.69%), while the best model, Gemini 2.0 Flash Thinking, reached 72.03%. However, the study covers only large listed companies and depends on closed-source models such as GPT-4o mini.

Study (4)

AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems

This study, by Nacar et al. (Tuwaiq Academy, ArabicNLP 2026), starts from the problem that Arabic tool use is still insufficiently evaluated, as most tool-calling benchmarks focus on English.

The researchers organize AISA-ArabicFC, the first shared task on Arabic function calling. For each Arabic request, the system must decide whether to call a tool, choose the correct function and extract its arguments, or otherwise abstain.

The dataset has 12,220 requests in five dialect groups, including Levantine (16.0%), across eight domains such as banking and finance, with 27 tools. Systems were evaluated by function-name accuracy and argument exact match, with a separate analysis per dialect.

Twenty-one teams submitted valid results, and a LoRA-adapted 4B model ranked first (0.9279), ahead of the zero-shot GPT-4o baseline (0.8195). Argument extraction, not tool selection, was the main challenge, and one team found Levantine to be its weakest dialect, with most failures caused by date-format and enumeration patterns. However, the task covers single-turn requests with only one call, so it does not test multi-step plans.

Study (5)

Learning to Ask: When LLM Agents Meet Unclear Instruction

This study, by Wang et al. (Renmin University of China and The Chinese University of Hong Kong, EMNLP 2025), starts from the problem that tool-using agents assume user instructions are always clear. When information is missing, agents tend to invent the missing arguments, which can cause hallucinations and risks.

The researchers propose Ask-when-Needed (AwN), which makes the agent check each instruction before calling a tool, and ask the user whenever something is unclear.

From 1,000 real user queries, they found four types of unclear instructions: missing key information (56.0%), multiple references, errors, and requests beyond the tools. They built NoisyToolBench with 200 such instructions, and an automatic evaluator (ToolEvaluator) that plays the role of the user.

Tested on 8 LLMs, AwN raised GPT-4o's rate of correct clarifying questions from 0.52 to 0.90 for instructions with missing information. However, the authors note that a big gap to perfect performance remains, and that the automatic evaluation is not fully accurate.

Study (6)

Aligning LLM Agents by Learning Latent Preference from User Edits (CIPHER)

This study, by Gao et al. (Cornell University and Microsoft Research, NeurIPS 2024), starts from the problem that users often edit an agent's responses to match their own preferences. Fine-tuning the model for every user is costly and hard to scale.

The researchers propose CIPHER, which learns a written description of the user's preference from their edits and adds it to the prompt, while the model itself stays unchanged.

For each new request, CIPHER retrieves the most similar past requests using embeddings and combines their learned preferences. It was tested on summarization and email writing with a GPT-4 simulated user over 200 rounds, measuring the edit distance of the user's changes.

CIPHER reduced user edits by 31% in summarization and 73% in email writing compared with no learning, and won 73.3% of human comparisons against the best baseline. However, a clear gap remains compared with knowing the true preference, and the main evaluation used a simulated user.

Comparison of Previous Studies

No	Study	Main Idea	Techniques	Dataset	Architecture	Evaluation Metrics	Results
1	ReAct (ICLR 2023)	Interleave reasoning and acting so the model can use external tools	Few-shot prompting of PaLM-540B, Wikipedia API, fine-tuning of PaLM-8B/62B	HotpotQA, FEVER, ALFWorld, WebShop	Thought–action–observation loop	Exact match, accuracy, success rate, score	+34% (ALFWorld) and +10% (WebShop) absolute success over imitation/RL methods; FEVER 60.9 vs. 56.3 for CoT; 0% of failures from hallucination vs. 56% for CoT
2	TinyAgent (EMNLP 2024)	Small local models that perform function calling at the edge	LLMCompiler, LoRA fine-tuning, Tool RAG (DeBERTa-v3-small), 4-bit quantization	80K synthetic examples generated by GPT-4-Turbo, 16 Mac functions	Local function-calling planner (DAG) with tool retrieval	Success rate (DAG match), latency, model size	1.1B: 80.06% and 7B: 84.95% vs. GPT-4-Turbo 79.08%; 1.1B at 4-bit: 0.68 GB, 2.9 s

No	Study	Main Idea	Techniques	Dataset	Architecture	Evaluation Metrics	Results
3	FinStat2SQL (INLG 2025)	Natural-language queries over Vietnamese financial statements	Entity extraction, vector search, few-shot SQL, self-correction, LoRA, DPO/KTO	Database of 200 companies; 12,400 synthetic SQL samples; ~300 questions with 821 MCQs	Multi-agent Text2SQL pipeline combining large and small models	Accuracy via LLM-judged MCQs, latency	7B: 61.33% vs. GPT-4o mini 59.14%; best overall Gemini 2.0 Flash Thinking 72.03%; under 4 s latency
4	AISA-ArabicFC (ArabicNLP 2026)	First shared task on Arabic function calling	LoRA fine-tuning, prompting, ensembles, TF-IDF baseline	12,220 Arabic requests, 5 dialect groups, 8 domains, 27 tools	One call or abstention among 4 candidate tools	FnAcc, ArgEM, ThinkRate, per-dialect score	Winner Qwen3-4B + LoRA: 0.9279; ArgEM peak 0.8816; zero-shot GPT-4o 0.8195
5	Learning to Ask (EMNLP 2025)	Agent asks the user when instructions are unclear	Ask-when-Needed prompting on CoT and DFSDT, ToolEvaluator (GPT-4o judge)	NoisyToolBench: 200 modified ToolBench instructions, 4 issue types	Clarification step before each API call	A1, A2, A3, redundant questions, steps	GPT-4o + CoT: A1 0.52 → 0.90 (missing information), 0.10 → 0.92 (beyond tool capabilities)
6	CIPHER (NeurIPS 2024)	Learn user preferences from edits without fine-tuning	Embedding retrieval (MPNET/BERT), LLM preference inference and aggregation, GPT-4	Summarization and email writing with a GPT-4 simulated user, 200 rounds	Frozen LLM with a preference history retrieved per context	Cumulative edit distance, preference accuracy, token cost	Edits reduced by 31% (summarization) and 73% (email); 73.3% human win rate vs. the best baseline

Research Gap

The analysis of previous studies shows that each study focused on only one aspect of LLM agents. ReAct and TinyAgent built tool-using agents, but not for Arabic or for finance. FinStat2SQL worked on finance, but on company financial statements rather than household expenses.

AISA-ArabicFC evaluated Arabic tool calling, but only single requests with one call. Learning to Ask and CIPHER improved how agents deal with users, by asking questions and learning from edits, but outside Arabic and outside the financial domain.

Therefore, there is still a lack of systems that combine:

- A tool-calling agent that can plan several actions for a single request
- Arabic dialects, including the Syrian dialect

- Family budget and expense management
- Asking the user when a request is unclear
- Learning from the user's corrections over time

Contribution of the Proposed System

The previous studies show several important ideas that can be used in developing the Family Budget Agent project.

First, ReAct shows that combining reasoning with tool actions reduces hallucination (0% of its failures vs. 56% for chain-of-thought). **In our project**, it is the basis of the agent loop that replaces the fixed pipeline of Graduation Project 1.

Second, TinyAgent shows that function-calling accuracy depends strongly on how a model is adapted to the task: fine-tuning raised its 1.1B model from 12.71% to 78.89% success, and Tool RAG brought it to 80.06%, above GPT-4-Turbo (79.08%). **In our project**, the cloud models are used without fine-tuning, but a large and a smaller model are compared inside the same agent, and success is measured over the whole sequence of tool calls, as TinyAgent measures its whole plan.

Third, FinStat2SQL shows that financial questions can be answered from a database through SQL, that model size matters (its fine-tuned 7B model reached 61.33%, while Llama 3.3 70B reached 65.69%), and that example data must be checked against the evaluation data. **In our project**, totals and budgets are computed by SQL inside the tools, and the examples placed in the prompt are checked for overlap with the human-written test set.

Fourth, AISA-ArabicFC shows that argument extraction, not tool selection, is the main challenge in Arabic function calling, and one team traced most of its Levantine errors to dates and enumerated values. **In our project**, we evaluate both tool selection and argument accuracy, especially for amounts, dates and categories in the Syrian dialect.

Fifth, Learning to Ask shows that prompting the agent to ask raised GPT-4o's correct clarifying questions from 0.52 to 0.90. **In our project**, the agent asks when the amount, category or currency is unclear, and our unclear test cases follow the same four types.

Finally, CIPHER shows that retrieving what was learned from similar past interactions reduces user edits by 31% to 73% without fine-tuning. **In our project**, user corrections are stored in ChromaDB and retrieved for similar future messages.

Based on these results, our project will combine these techniques in one agent able to:

- Understand expense messages and questions in the Syrian dialect and decide which tools to call
- Execute several tool calls for a single message (record expenses, compute totals, check budgets)
- Compute all numbers through SQL queries instead of generating them with the language model
- Ask the user for clarification when the amount, category or currency is unclear
- Store the user's corrections and retrieve similar ones to improve future answers
- Compare Llama 3.3 70B with the smaller Llama 3.1 8B, and count failed API requests instead of excluding them

References

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
2. Erdogan, L. E., Lee, N., Jha, S., Kim, S., Tabrizi, R., Moon, S., Hooper, C., Anumanchipalli, G., Keutzer, K., & Gholami, A. (2024). TinyAgent: Function Calling at the Edge. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 80–88.
3. Nguyen, Q. H., Trinh, P. A., Mai, P. Q. H., & Trinh, T. P. (2025). FinStat2SQL: A Text2SQL Pipeline for Financial Statement Analysis. Proceedings of the 18th International Natural Language Generation Conference (INLG), pp. 449–464.
4. Nacar, O., Alzubaidi, W., & Alkhalifah, M. (2026). AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems. Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026), Volume 2: Shared Tasks, pp. 1531–1546.
5. Wang, W., Shi, J., Ling, Z., Chan, Y.-K., Wang, C., Lee, C., Yuan, Y., Huang, J., Jiao, W., & Lyu, M. R. (2025). Learning to Ask: When LLM Agents Meet Unclear Instruction. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 21773–21784.
6. Gao, G., Taymanov, A., Salinas, E., Mineiro, P., & Misra, D. (2024). Aligning LLM Agents by Learning Latent Preference from User Edits. 38th Conference on Neural Information Processing Systems (NeurIPS 2024).